



UNIVERSITY OF  
TORONTO

# Applied Deep Learning

## Lecture 2: *Deep NNs and Gradient Descent*

Ali Bereyhi

ECE Department, University of Toronto

Fall 2026



UNIVERSITY OF  
TORONTO

# Lecture Overview

## Quick Recap: *ML Components*

Any ML problem has *three components*:

1. *Dataset* which is the collection of samples

$$\boxed{\text{supervised learning}} \quad \mathbb{D} = \{(\mathbf{x}_i, y_i) : i = 1, \dots, I\}$$

2. *Model* that describes the relation between the *input* and *label*

$\mathbf{h}$ : Hyperparameters

$$z = f_{\mathbf{h}}(\mathbf{x} | \mathbf{w})$$

$\mathbf{w}$ : Learnable Parameters

3. *Loss* quantifies the difference between the *model's output* and *true label*

$$\mathcal{L}(z_i, y_i)$$

## Recap: *Training*

The *training* is performed by minimizing the empirical risk

$$\begin{aligned}\mathbf{w}^{\star} &= \underset{\mathbf{w}}{\operatorname{argmin}} \hat{R}(\mathbf{w}) && \text{(Training)} \\ &= \underset{\mathbf{w}}{\operatorname{argmin}} \frac{1}{I} \sum_{i=1}^I \mathcal{L}(f_{\mathbf{h}}(\mathbf{x}_i | \mathbf{w}), y_i)\end{aligned}$$

## Lecture 2. *Deep NNs and Gradient Descent*

Today, we are going to learn

1. Universal approximation by neural networks
  - *NNs can approximate any function if they are **deep enough!***
2. Deep NNs and in particular **multi layer perceptrons**
  - *MLPs are the **most basic** form of NNs*
3. Gradient descent algorithm
  - *It helps us minimize the **empirical risk***



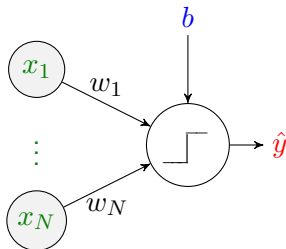
UNIVERSITY OF  
TORONTO

# Universal Approximation

## Training Perceptron Machine: *ML 1*

1. Dataset  $\mathbb{D} = \{(\mathbf{x}_i, y_i) : i = 1, \dots, I\}$  with binary labels  $y_i \in \{0, 1\}$
2. Perceptron as the model

$$\hat{y} = s(\mathbf{w}^T \mathbf{x} + b) = \begin{cases} 1 & \text{if } \mathbf{w}^T \mathbf{x} + b \geq 0 \\ 0 & \text{if } \mathbf{w}^T \mathbf{x} + b < 0 \end{cases}$$



3. Error indicator as the loss function

$$\mathcal{L}(\hat{y}, y) = \mathbb{1}(\hat{y} \neq y) = \begin{cases} 1 & \hat{y} \neq y \\ 0 & \hat{y} = y \end{cases}$$

## Training Perceptron Machine: *ML 2*

Let's write down the *empirical risk*

$$\begin{aligned}\hat{R}(\mathbf{w}, b) &= \frac{1}{I} \sum_{i=1}^I \mathcal{L}(\hat{y}_i, y_i) \\ &= \frac{1}{I} \sum_{i=1}^I \mathbb{1}(\hat{y}_i \neq y_i) \\ &= \frac{1}{I} \sum_{i=1}^I \mathbb{1}(s(\mathbf{w}^\top \mathbf{x}_i + b) \neq y_i) \\ &= \frac{\text{\# of times perceptron misclassifies}}{I} \\ &\equiv \text{\textit{Error Rate}}\end{aligned}$$

## Training Perceptron Machine: *ML 3*

We should now minimize the *empirical risk*

$$\mathbf{w}^*, b^* = \underset{\mathbf{w} \in \mathbb{R}^N, b \in \mathbb{R}}{\operatorname{argmin}} \hat{R}(\mathbf{w}, b) = \underset{\mathbf{w} \in \mathbb{R}^N, b \in \mathbb{R}}{\operatorname{argmin}} \frac{1}{I} \sum_{i=1}^I \mathbb{1}(s(\mathbf{w}^\top \mathbf{x}_i + b) \neq y_i)$$

Let's see what Rosenblatt did: *Rosenblatt's perceptron had no bias, i.e.,  $b = 0$*

```
1: Start with  $\mathbf{w} = \mathbf{0}$  or some small random initial value
2: while  $\hat{R}(\mathbf{w}) \neq 0$  do
3:   for  $i = 1 : I$  do
4:     Compute  $z_i = \mathbf{w}^\top \mathbf{x}_i$  and  $\hat{y}_i = s(z_i)$       # pass through perceptron
5:     if  $\hat{y}_i \neq y_i$  then
6:        $\mathbf{w} \leftarrow \mathbf{w} - \operatorname{sgn}(z_i) \mathbf{x}_i$ 
7:     end if
8:   end for
9: end while
```

## Training Perceptron Machine: *Perceptron Algorithm*

```
1: Start with  $\mathbf{w} = \mathbf{0}$  or some small random initial value
2: while  $\hat{R}(\mathbf{w}) \neq 0$  do
3:   for  $i = 1 : I$  do
4:     Compute  $z_i = \mathbf{w}^T \mathbf{x}_i$  and  $\hat{y}_i = s(z_i)$       # pass through perceptron
5:     if  $\hat{y}_i \neq y_i$  then
6:        $\mathbf{w} \leftarrow \mathbf{w} - \text{sgn}(z_i) \mathbf{x}_i$ 
7:     end if
8:   end for
9: end while
```

- Outer loop stops only if  $\hat{R}(\mathbf{w}) = 0$  which is minimum empirical risk
- In inner loop, let's say at iteration  $t$  error occurs for  $\mathbf{x}_i$

↳ **either**  $z_i = \mathbf{w}_t^T \mathbf{x}_i > 0$  and  $y_i = 0$ : we update as  $\mathbf{w}_{t+1} \leftarrow \mathbf{w}_t - \mathbf{x}_i$

$$\mathbf{w}_{t+1}^T \mathbf{x}_i = (\mathbf{w}_t - \mathbf{x}_i)^T \mathbf{x}_i = \mathbf{w}_t^T \mathbf{x}_i - \|\mathbf{x}_i\|^2 \leq \mathbf{w}_t^T \mathbf{x}_i$$

so the algorithm pushes the weights somewhere that  $z_i$  could get *negative*

## Training Perceptron Machine: *Perceptron Algorithm*

```
1: Start with  $\mathbf{w} = \mathbf{0}$  or some small random initial value
2: while  $\hat{R}(\mathbf{w}) \neq 0$  do
3:   for  $i = 1 : I$  do
4:     Compute  $z_i = \mathbf{w}^T \mathbf{x}_i$  and  $\hat{y}_i = s(z_i)$       # pass through perceptron
5:     if  $\hat{y}_i \neq y_i$  then
6:        $\mathbf{w} \leftarrow \mathbf{w} - \text{sgn}(z_i) \mathbf{x}_i$ 
7:     end if
8:   end for
9: end while
```

- Outer loop stops only if  $\hat{R}(\mathbf{w}) = 0$  which is minimum empirical risk
- In inner loop, let's say at iteration  $t$  error occurs for  $\mathbf{x}_i$

↳ **or**  $z_i = \mathbf{w}_t^T \mathbf{x}_i < 0$  and  $y_i = 1$ : we update as  $\mathbf{w}_{t+1} \leftarrow \mathbf{w}_t + \mathbf{x}_i$

$$\mathbf{w}_{t+1}^T \mathbf{x}_i = (\mathbf{w}_t + \mathbf{x}_i)^T \mathbf{x}_i = \mathbf{w}_t^T \mathbf{x}_i + \|\mathbf{x}_i\|^2 \geq \mathbf{w}_t^T \mathbf{x}_i$$

so the algorithm pushes the weights somewhere that  $z_i$  could get *positive*

## Training Perceptron Machine: *Perceptron Algorithm*

```
1: Start with  $\mathbf{w} = \mathbf{0}$  or some small random initial value
2: while  $\hat{R}(\mathbf{w}) \neq 0$  do
3:   for  $i = 1 : I$  do
4:     Compute  $z_i = \mathbf{w}^T \mathbf{x}_i$  and  $\hat{y}_i = s(z_i)$       # pass through perceptron
5:     if  $\hat{y}_i \neq y_i$  then
6:        $\mathbf{w} \leftarrow \mathbf{w} - \text{sgn}(z_i) \mathbf{x}_i$ 
7:     end if
8:   end for
9: end while
```

- + *It makes sense that in each iteration  $\mathbf{w}$  gets modified towards a **right direction**! But can we guarantee that this algorithm always converges?*
- Well! Let's try some examples

## Perceptron Algorithm: *AND Operator*

Assume that we have the following **dataset** two-dimensional **inputs**

$$\mathbb{D} = \left\{ \left( \begin{bmatrix} 0 \\ 0 \end{bmatrix}, 0 \right), \left( \begin{bmatrix} 0 \\ 1 \end{bmatrix}, 0 \right), \left( \begin{bmatrix} 1 \\ 0 \end{bmatrix}, 0 \right), \left( \begin{bmatrix} 1 \\ 1 \end{bmatrix}, 1 \right) \right\}$$

We intend to **train perceptron** with this **dataset** via **perceptron algorithm**

---

*Before we start **training**, we note that this dataset represents the **AND operator***

$$\forall (\mathbf{x}_i, y_i) \in \mathbb{D} : y_i = x_{i,1} \wedge x_{i,2}$$

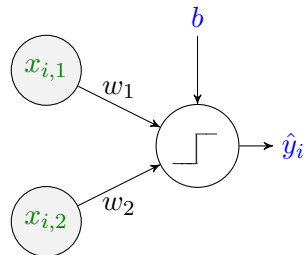
*so, we basically want to see, if we could realize this operator via **perceptron***

# Perceptron Algorithm: AND Operator

```

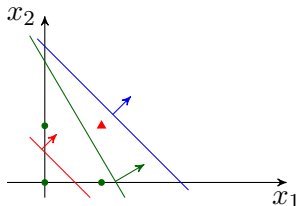
1: Start with  $\mathbf{w} = [1.1, 1.1]^T$  and  $b = -2.25$ : blue
2: while  $\hat{R}(\mathbf{w}, b) \neq 0$  do
3:   for  $i = 1 : I$  do
4:     Compute  $z_i = \mathbf{w}^T \mathbf{x}_i + b$  and  $\hat{y}_i = s(z_i)$ 
5:     if  $\hat{y}_i \neq y_i$  then
6:        $\mathbf{w} \leftarrow \mathbf{w} - \text{sgn}(z_i) \mathbf{x}_i$  and  $b \leftarrow b - \text{sgn}(z_i)$ 
7:     end if
8:   end for
9: end while

```



Let's show data-points with  $y_i = 1$  by  $\blacktriangle$  and those with  $y_i = 0$  by  $\bullet$

- Updated by  $\mathbf{x}_i = [1, 1]^T$   
 $\hookrightarrow \mathbf{w} = [2.1, 2.1]^T$  and  $b = -1.25$ : red
- Updated by  $\mathbf{x}_i = [0, 1]^T$   
 $\hookrightarrow \mathbf{w} = [2.1, 1.1]^T$  and  $b = -2.25$ : green



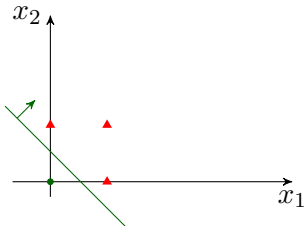
## Perceptron Algorithm: *AND* Operator

Great! We *trained* the *perceptron* to behave as *logical AND*

- + What about *logical OR*?
- Easy! Let's write the *dataset*

$$\mathbb{D} = \left\{ \left( \begin{bmatrix} 0 \\ 0 \end{bmatrix}, 0 \right), \left( \begin{bmatrix} 0 \\ 1 \end{bmatrix}, 1 \right), \left( \begin{bmatrix} 1 \\ 0 \end{bmatrix}, 1 \right), \left( \begin{bmatrix} 1 \\ 1 \end{bmatrix}, 1 \right) \right\}$$

Trying the *perceptron algorithm*, we end up with some *linear classifier* like

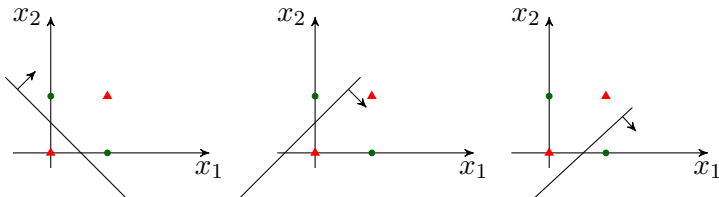


## Perceptron Algorithm: *False Conclusion*

A false initial conclusion is that *perceptron* can realize *any* binary function! But, it is easy to see that *this is not the case!* Let's consider **XOR**:

$$x_1 \oplus x_2 = \begin{cases} 1 & \text{if } x_1 \neq x_2 \\ 0 & \text{if } x_1 = x_2 \end{cases} \rightsquigarrow \mathbb{D} = \left\{ \left( \begin{bmatrix} 0 \\ 0 \end{bmatrix}, 0 \right), \left( \begin{bmatrix} 0 \\ 1 \end{bmatrix}, 1 \right), \left( \begin{bmatrix} 1 \\ 0 \end{bmatrix}, 1 \right), \left( \begin{bmatrix} 1 \\ 1 \end{bmatrix}, 0 \right) \right\}$$

It's not hard to see that *perceptron* cannot learn this function



## Multi-Layering Perceptrons

Let's play a bit with *logical XOR*: recall that

$$x_1 \oplus x_2 = \begin{cases} 1 & \text{if } x_1 \neq x_2 \\ 0 & \text{if } x_1 = x_2 \end{cases}$$

We can write this function as

$$x_1 \oplus x_2 = (x_1 \vee x_2) \wedge (\bar{x}_1 \vee \bar{x}_2)$$

where we have used the following notation

$$\bar{x} \equiv \text{complement of } x = 1 - x \quad \text{and} \quad \vee \equiv \text{logical OR}$$

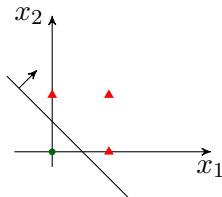
# Multi-Layering Perceptrons

*Logical XOR expands as*

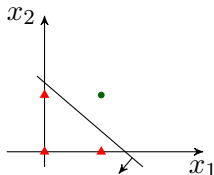
$$y = x_1 \oplus x_2 = \underbrace{(x_1 \vee x_2)}_{h_1} \wedge \underbrace{(\bar{x}_1 \vee \bar{x}_2)}_{h_2} = h_1 \wedge h_2$$

We can learn  $h_1$  and  $h_2$  by two different *perceptrons*

$$h_1 = x_1 \vee x_2$$

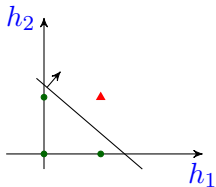


$$h_2 = \bar{x}_1 \vee \bar{x}_2 \equiv \overline{x_1 \wedge x_2}$$

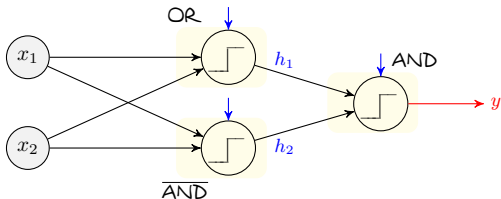


## Multi-Layering Perceptrons

We can further learn  $y = h_1 \wedge h_2$  by another *perceptron*

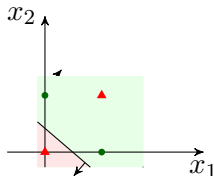


Well! It's true that we cannot learn **XOR** by a single *perceptron*, but we can learn it with a *network of three perceptrons*!



## Multi-Layering Perceptrons: Geometrical Interpretation

Let's see geometrically what this network of *perceptrons* does



- First *perceptron* classifies
- Second *perceptron* classifies
- Third *perceptron* intersects the two regions

## Multi-Layering Perceptrons: *Correct Conclusion*

We can learn *any* binary function using a *network of perceptrons*!

This has given birth to the idea of *neural network*

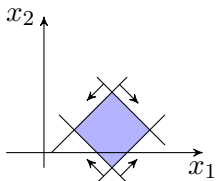
### Neural Networks: *Intuitive Introduction*

looking at *perceptron* as an *artificial neuron*, we can *learn* any complicated function when we have a network of these *neurons*  $\equiv$  a *Neural Network*

- + *But, why should we care only about binary functions? Don't we learn other type of functions as well?*
- Well! We can extend the idea to other problems as well!

## Multi-Layering Perceptrons: *More Complicated Functions*

Assume we have *binary classification* with *real-valued inputs*: we want to train a classifier that distinguishes between the *two-dimensional points* inside the *blue area* and outside of it



- We classify *Region 1* with three *perceptrons*
- We classify *Region 2* with three *perceptrons*
- We *intersect* the two *regions* with a *perceptron*

*A network of 7 *perceptrons* is more than enough!*

## Multi-Layering Perceptrons: *More Complicated Functions*

- + What if the classification problem is *not binary*?
- Well! We already have seen that we can reduce a **multi-class** classification to a series of binary classifications<sup>1</sup>

We want to classify an image as *dog, cat or car*

- ▶ *Binary Classification 1: Is it class 0: dog or class 1: {cat, car}?*
  - ↳ If *class 0*  $\rightsquigarrow$  classification ended
  - ↳ If *class 1*  $\rightsquigarrow$  *Binary Classification 2: Is it class 0: cat or class 1: car?*

### Moral of Story

We can learn *any classifier* with *high accuracy* using a network of *perceptrons*

<sup>1</sup> There are better **multi-class** classification techniques that we learn later

## Multi-Layering Perceptrons: *More Complicated Functions*

- + *Fair enough! We are happy with classification! But, what about the case that we have **real-valued labels**? Can we do it by **perceptrons**?*
- This is the **regression problem**!
- And, Yes! We can do this as well using **perceptrons**

### Regression

Regression is a **supervised** learning problem in which

***labels** belong to a continuous set, e.g.,  $y_i \in \mathbb{R}$*

the **label**  $y_i$  in this case is the function sample at  $x_i$

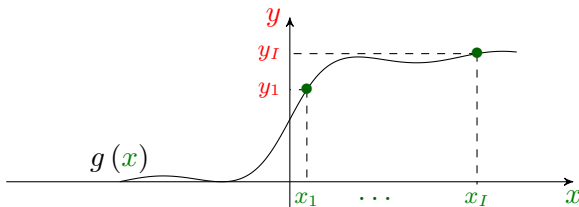
## Multi-Layering Perceptrons: *Regression*

In *regression*, we learn a *real-valued function*

Let's look at a simple case with *one-dimensional inputs*, where we can visualize

$$\mathbb{D} = \{(x_i, y_i) : i = 1, \dots, I\}$$

A visualization of this *dataset* is



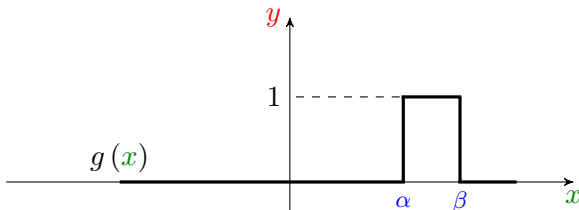
and we want to *learn*  $g(\cdot)$

## Multi-Layering Perceptrons: *Regression*

The main question is

can we realize an *arbitrary*  $g(\cdot)$  via a network of *perceptrons*?

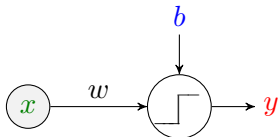
Let's start with a simple function: *the unit pulse*



We can realize this function using three *perceptrons*

## Multi-Layering Perceptrons: *Regression*

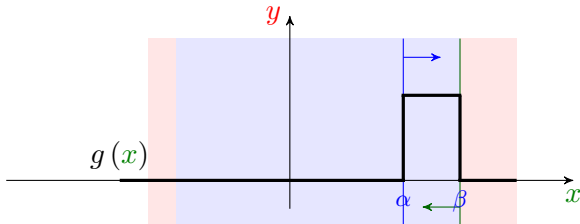
First, let's recall how **perceptron** looks with **one-dimensional** input



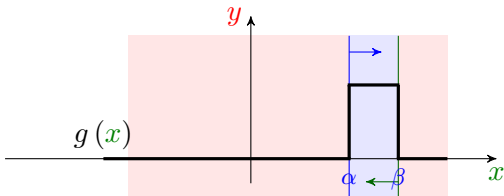
↳ with  $w = 1$  and  $b = -\alpha$ : **blue**

↳ with  $w = -1$  and  $b = \beta$ : **green**

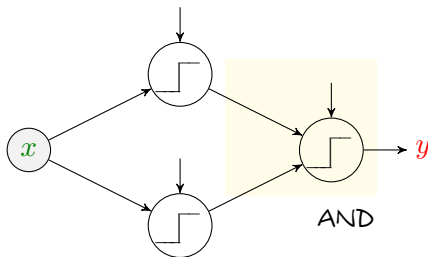
We realize the pulse by applying **AND** on the outputs of these **perceptrons**



# Multi-Layering Perceptrons: *Regression*

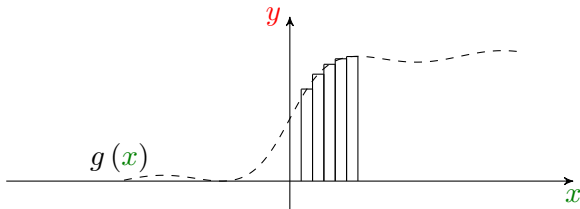


So, we can realize the unit pulse via the following *network of perceptrons*



## Multi-Layering Perceptrons: *Regression*

We can approximate a *general* function via a *weighted sum* of unit pulses



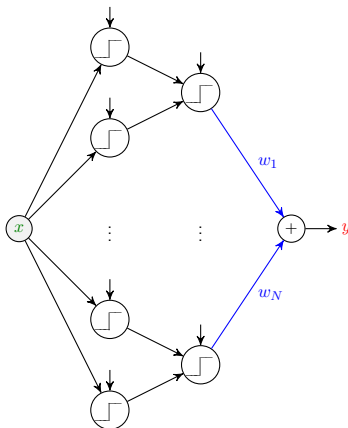
This means that we can approximate *any* function by

- realizing each pulse by a *network of perceptrons*
- applying *linear transform* on the *outputs* of these *networks*

## Multi-Layering Perceptrons: *Regression*

*Real-valued functions* are *well-approximated* by

a network of *perceptrons* + a *linear transform*



## Multi-Layering Perceptrons: *Summary*

A network of *perceptrons* seems to be a very sophisticated *model*

- It can *learn any classifier*
- It can *approximate any function* with *arbitrary accuracy*

### Universal Approximation Theorem (*informal*)

Given function  $g(\cdot)$  and  $\varepsilon > 0$ , there exists a *neural network*  $f_{\mathbf{h}}(\cdot|\mathbf{w})$  that

$$\sup_{\mathbf{x}} \|g(\mathbf{x}) - f_{\mathbf{h}}(\mathbf{x}|\mathbf{w})\| \leq \varepsilon$$

So, it seems natural to train them for our learning tasks

*A network of *perceptrons* is a special *artificial neural network**

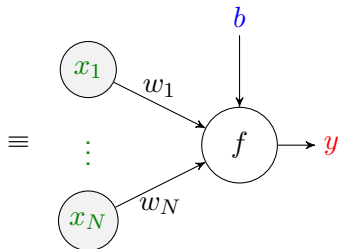
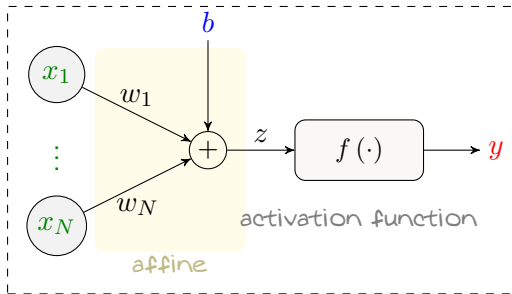


UNIVERSITY OF  
TORONTO

# Deep Neural Networks

## Artificial Neuron

Perceptron is a *special* artificial neuron. In general, an artificial neuron is



A neuron with an  $N$ -dimensional input has  $N + 1$  learnable parameters

- $N$  weights, i.e.,  $w_1, \dots, w_N$
- a bias

## Artificial Neuron

The output of neuron  $y$  is related to its inputs as  $x$

$$y = f(\mathbf{w}^T \mathbf{x} + b)$$

where we define

- $\mathbf{w}^T = [w_1, \dots, w_N]$  to be the vector of weights
- $b$  to be the bias
- $f(\cdot) : \mathbb{R} \mapsto \mathbb{R}$  to be the activation function

A neuron can in general have any activation

- In perceptron, the activation function was step function<sup>2</sup>
- Linear activation  $f(z) = z$  reduces the neuron to basic linear model

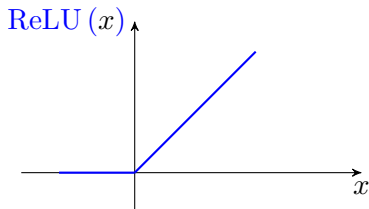
---

<sup>2</sup>It turns out soon that this was in fact a bad choice of activation!

## Artificial Neuron: *Activation*

If we intend to learn *nonlinear* functions; then, we need *nonlinear activation*

Some sample activation functions: *rectified linear unit (ReLU) function*

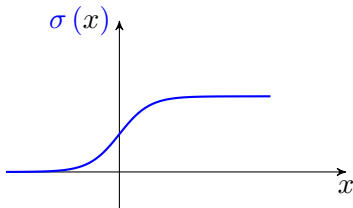


$$\text{ReLU}(x) = \max\{x, 0\}$$

## Artificial Neuron: *Activation*

If we intend to learn *nonlinear* functions; then, we need *nonlinear activation*

Some sample activation functions: *sigmoid* function

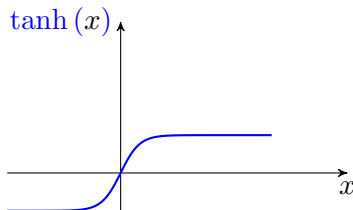


$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

## Artificial Neuron: *Activation*

If we intend to learn *nonlinear* functions; then, we need *nonlinear activation*

Some sample activation functions: *hyperbolic tangent* function

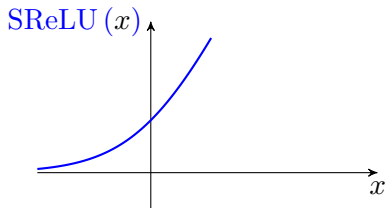


$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

## Artificial Neuron: *Activation*

If we intend to learn *nonlinear* functions; then, we need *nonlinear activation*

Some sample activation functions: *soft ReLU* function

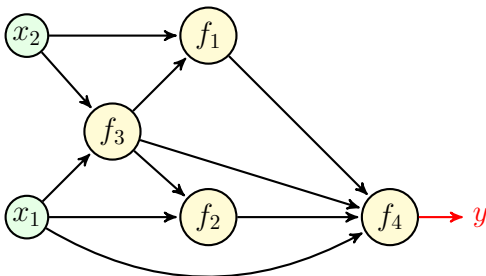


$$\text{SReLU}(x) = \log(1 + e^x)$$

# Neural Network

## Artificial Neural Network

Artificial neural network is a **directed graph** that connects a set of **inputs** to a set of **outputs**: the nodes of this graph are **neurons** whose **activation functions** are known and whose weights and **biases** are **learnable**

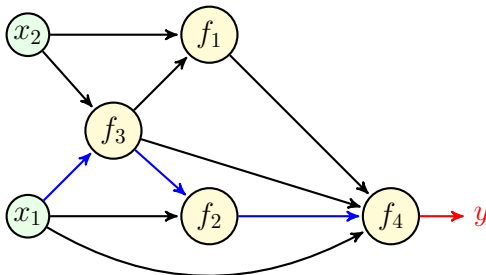


Unless we are talking to biologists, we can safely drop the term **artificial** 😊

## Neural Network: *Depth*

### Depth of a Neural Network

The longest path between an *input* and the *output*

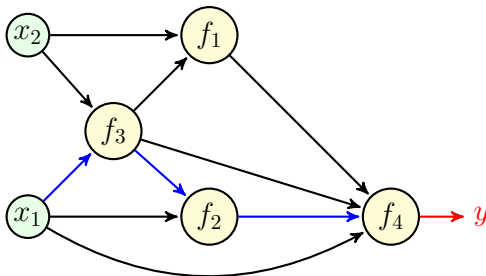


Here, the depth is **3**

# Deep Neural Network

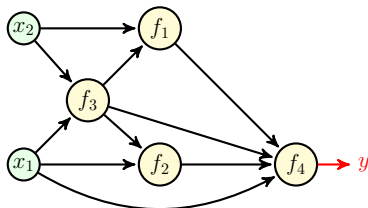
## Deep Neural Network

A neural network whose *depth* is *larger than 2*



Here, the depth is  $3 > 2$   $\rightsquigarrow$  it's a deep neural network

## Neural Networks are *Models*



Here, we've chosen to have **4 neurons** with **activations**  $f_1(\cdot), \dots, f_4(\cdot)$  arranged in the **above form**: *these are hyperparameters*

Now that the **architecture** is fixed, we could say

- ↳ **Neurons 1, 2, 3** have two **inputs**: each of them has two weights and a **bias**
- ↳ **Neurons 4** has four **inputs**: it has four weights and a **bias**

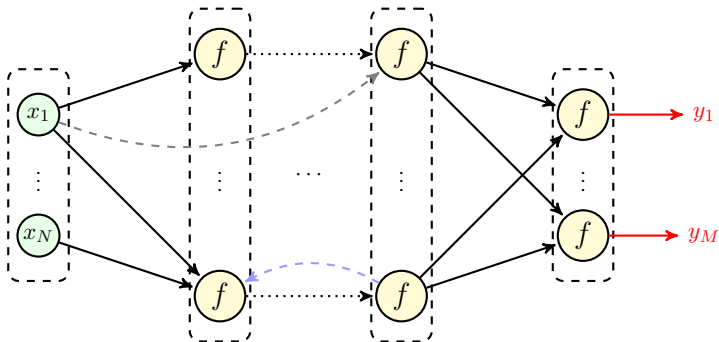
So in total we have  $3 \times (2 + 1) + (4 + 1) = 14$  **learnable** parameters

## Neural Network: *Layered Architecture*

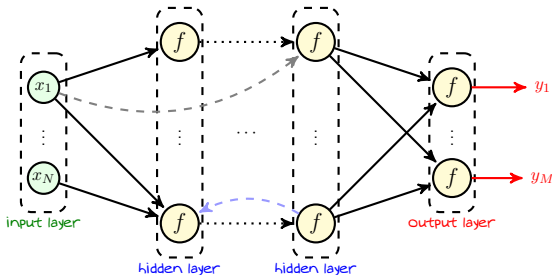
In practice, we use *neural networks* with *layered architectures*

### Layer

A subset of *neurons* that are in the same distance from *inputs*



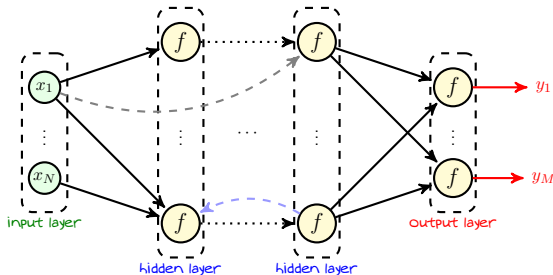
## Neural Network: *Layered Architecture*



There are three types of layers

- *Input layer* that only contains inputs **Attention: No neuron here!**
- *Output layer* that contains the neurons computing network outputs
- *Hidden layers* that contain neurons and are in between

## Neural Network: *Layered Architecture*



It's readily seen from the definition that with **layered architecture**

$$\text{depth} = \# \text{ hidden layers} + 1$$

Hence, we could equivalently say that

a *deep* neural network has *more than one hidden layer*

## Various Layered Architectures

Based on **information flow**, we could have

1. *Feedforward NNs* in which **information** only moves **forward**
  - ↳ *Multi-Layer Perceptrons* abbreviated as *MLPs*<sup>3</sup>
  - ↳ *Convolutional NNs* abbreviated as *CNNs*
2. *Recurrent NNs* in which **information** can **circulate**

We start with **MLPs** which are known to be

*vanilla NNs*,

*i.e., the **most basic** architecture we could think for an NN*

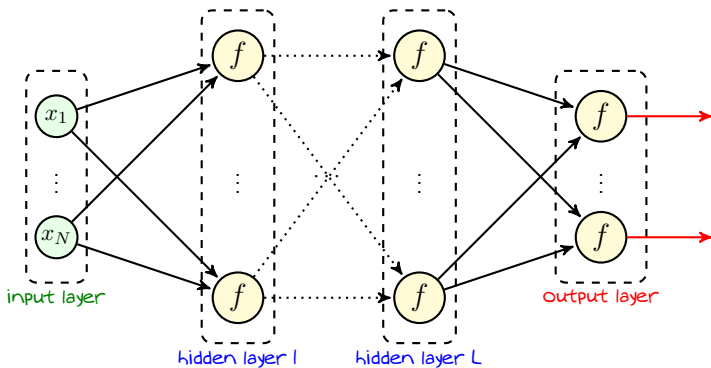
---

<sup>3</sup> You may say that this is a *misnomer* and you are right! Check the [wikipedia page](#)

## MLPs $\equiv$ Fully-Connected FNNs

### Fully-Connected

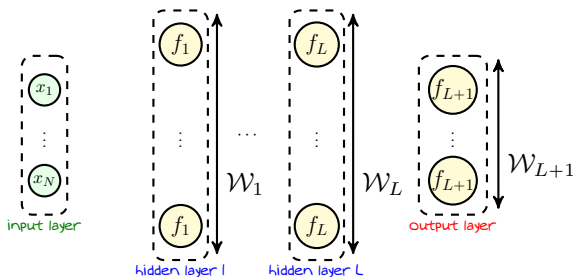
*In an MLP, each node is connected to all nodes in the next layer*



## MLPs: *Layer Width*

### Width of a Layer

The width of layer  $\ell$  is the number of neurons in layer  $\ell$



*Some people call the largest width, the width of the network, i.e.,*

$$\mathcal{W} = \max_{\ell \in \{1, \dots, L+1\}} \mathcal{W}_\ell$$

## MLPs as Models

We want to use an *MLP* with *ReLU activated* neurons

- To write down the model, we need to know

↳ *the number of hidden layers  $L$*

↳ *width of each layer  $\mathcal{W}_\ell$*

these are the *hyperparameters*

- If we set the  $L$  and  $\mathcal{W}_\ell$ , we can specify the *learnable parameters*

↳ in *hidden layer 1*, we have  $\mathcal{W}_1$  neurons each having  $N$  weights and a bias

↳ in *hidden layer 2*, we have  $\mathcal{W}_2$  neurons each having  $\mathcal{W}_1$  weights and a bias

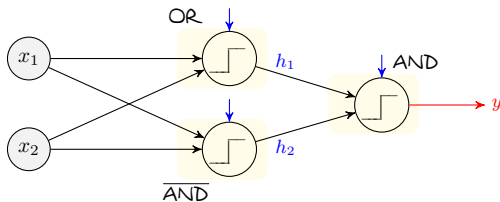
⋮

↳ in *output layer*, we have  $\mathcal{W}_{L+1}$  neurons each having  $\mathcal{W}_L$  weights and a bias

$$\# \text{ model parameters} = (N + 1) \mathcal{W}_1 + \sum_{\ell=1}^L (\mathcal{W}_\ell + 1) \mathcal{W}_{\ell+1}$$

## Example: *XOR Net*

Let's try our knowledge on the *XOR* neural network



- Input layer has two inputs
- Output layer has a single neuron
- One hidden layer with width two
  - ↳ It's a *shallow* NN

# Deep Learning

## Deep Learning

When we use a *deep neural network* to address a learning task,  
we are doing *deep learning*

Now, let's get things a bit clear

- In **ML**, we talk about any **model**, any **loss**, any **dataset**
- In **Representation Learning**, we roughly talk about **models** that can describe nonlinear **functions**: *this includes deep neural networks*
- In **DL**, we have a **deep neural network** as the **model**

So, we can say

$$\text{DL} \subset \text{Representation Learning} \subset \text{ML}$$

## Quick Summary

*At this point, we know*

- We can specify **dataset**, **model** and **loss**
- If we do DL, we use **deep neural networks** as **models**
- *Training?! . . .*

*We yet don't know*

- How to minimize the **empirical risk?**
  - ↳ We need an **algorithm**

*This is what we learn next!*



UNIVERSITY OF  
TORONTO

# Gradient Descent Algorithm

## Function Optimization

The *model training* always reduces to an optimization problem

$$\mathbf{w}^* = \underset{\mathbf{w}}{\operatorname{argmin}} \hat{R}(\mathbf{w}) = \underset{\mathbf{w}}{\operatorname{argmin}} \frac{1}{I} \sum_{i=1}^I \mathcal{L}(f_{\mathbf{h}}(\mathbf{x}_i | \mathbf{w}), y_i) \quad (\text{Training})$$

Let's recall each component of this optimization problem

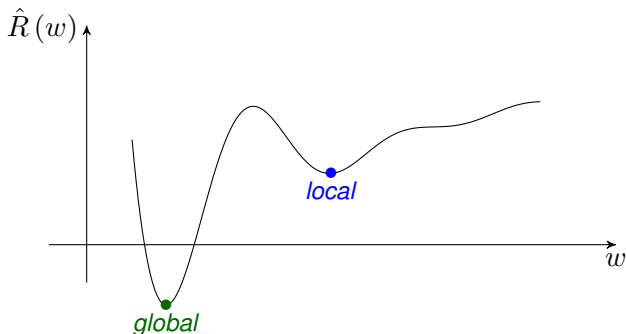
- $f_{\mathbf{h}}(\cdot | \mathbf{w})$  model with **hyperparameters**  $\mathbf{h}$  and **learnable parameters**  $\mathbf{w}$
- $\mathbf{x}_i$  is a data-point with **label**  $y_i$ , and  $I$  is size of **dataset**
- $\mathcal{L}$  is the loss function

No matter what we choose, at the end of the day we need to solve

$$\min_{\mathbf{w}} \hat{R}(\mathbf{w})$$

## Function Optimization

In general, the **empirical risk**  $\hat{R}(w)$  can have *local* and *global* minima



We would be delighted to meet the *global*; but, in practice we are happy with *local*

## Function Optimization

- + *Why is it a big problem? We could grid  $w$  and search on the grid*
- For only **one parameter** yes! But, **deep neural networks** have too many **too many parameters**!

*Say for an **accurate approximation** with only one parameter, we need  $G$  grids*

If we have  $D$  parameters, i.e.,  $\mathbf{w} \in \mathbb{R}^D$ , we need

$G^D$  grids

to get an approximation with the same accuracy!

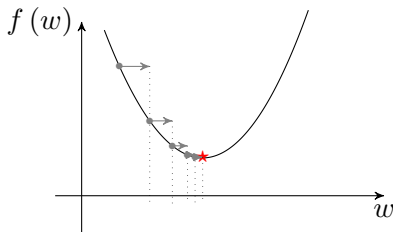
*For practical neural networks with  $D = 10^5$ , this is **impossible**!*

*we need to have an **optimization algorithm** with **feasible complexity***

## Optimization Algorithms $\equiv$ *Optimizer*

We look for an **optimization algorithm**, or as ML people call it “an **optimizer**”

- it starts from an **initial point** and moves for some steps
- in each step, it moves towards where the **empirical risk** is minimized
- it moves for a **feasible number of steps**



## Optimization Algorithms $\equiv$ *Optimizer*

*Let's clear things up:* we are looking for an iterative approach as below

- 1: Initiate at some  $\mathbf{w}^{(0)} \in \mathbb{R}^D$  and deviation  $\Delta = +\infty$
- 2: Choose some small  $\epsilon$
- 3: **while**  $\Delta > \epsilon$  **do**
- 4: Determine a vector  $\boldsymbol{\mu}^{(t)} \in \mathbb{R}^D$  based on  $\hat{R}(\mathbf{w}) \leftarrow$  we need to figure out
- 5: Update weights as  $\mathbf{w}^{(t)} \leftarrow \mathbf{w}^{(t-1)} + \boldsymbol{\mu}^{(t)}$
- 6: Update the deviation  $\Delta = |\hat{R}(\mathbf{w}^{(t)}) - \hat{R}(\mathbf{w}^{(t-1)})|$
- 7: **end while**

*We would like to have following properties*

- $\hookrightarrow$  most of the time the **empirical risk reduces** after the update in line 5
- $\hookrightarrow$  the algorithm stops after a **feasible number of iterations**

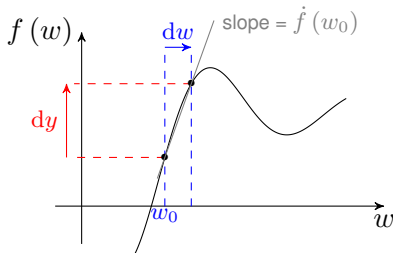
*We are going to get these properties if we set*

$\boldsymbol{\mu}^{(t)}$  to be **proportional** to the **negative of risk gradient** at  $\mathbf{w}^{(t-1)}$

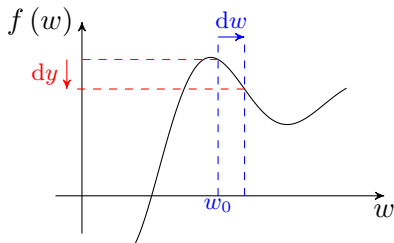
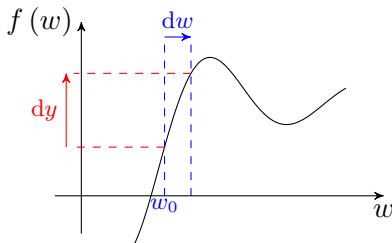
## Review: *Derivative of a Function*

Let  $y = f(w)$ . If we vary  $w$  around  $w_0$  with a tiny step  $dw$ ; then,

$$\text{Variation of } y = dy = \dot{f}(w_0) dw$$



## Review: *Derivative of a Function*



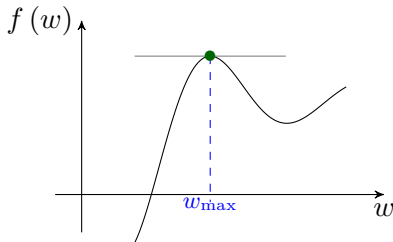
- $\dot{f}(w_0) > 0$  means *increasing  $w$  will increase  $y = f(w)$*
- $\dot{f}(w_0) < 0$  means *increasing  $w$  will decrease  $y = f(w)$*

### Key Observation

The derivative shows the **moving direction** on  $w$ -axis towards which the function **increases**; or alternatively, **its negative** is the **direction** that function **decreases**

## Review: *Derivative of a Function*

When do we have the derivative equal to zero? Either we are at a *maximum*



Starting before the maximum,

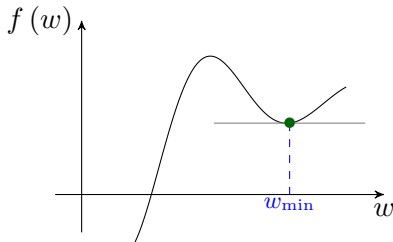
- The derivative is *first positive* and gradually *reduces to zero*
- As we pass the maximum the derivative gets *more and more negative*

So around the maximum as we *increase  $w$* , the derivative *reduces*

$$\ddot{f}(w_{\max}) = \frac{d^2}{dw^2} f(w_{\max}) = f''(w_{\max}) < 0$$

## Review: *Derivative of a Function*

When do we have the derivative equal to zero? Either we are at a *minimum*



Starting before the minimum,

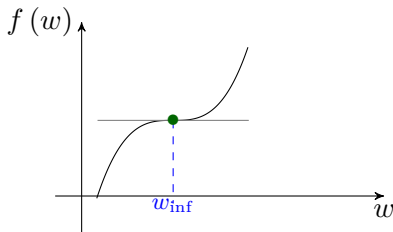
- The derivative is *first negative* and gradually *increases to zero*
- As we pass the minimum the derivative gets *more and more positive*

So around the maximum as we *increase  $w$* , the derivative *reduces*

$$\ddot{f}(w_{\min}) = \frac{d^2}{dw^2} f(w_{\min}) = f''(w_{\min}) > 0$$

## Review: *Derivative of a Function*

When do we have the derivative equal to zero? Either we are at an inflection



Starting before the inflection point,

- The derivative is *first positive* and gradually *decreases to zero*
- As we pass the inflection point the derivative gets *again positive*

So around the inflection point, the second derivative *changes sign*

$$\ddot{f}(w_{\text{inf}}) = \frac{d^2}{dw^2} f(w_{\text{inf}}) = f''(w_{\text{inf}}) = 0$$

## Review: Gradient of a Function

- + What about **multi-variable** functions, e.g.,  $f(\mathbf{w})$  for  $\mathbf{w} = [w_1, \dots, w_N]$ ?
- We can take derivative with respect to each variable, i.e.,

$$\dot{f}_n(\mathbf{w}_0) = \frac{\partial}{\partial w_n} f(\mathbf{w}_0)$$

This is what we call **partial derivative**

Let  $y = f(\mathbf{w})$ . If we vary  $\mathbf{w}$  around  $\mathbf{w}_0$  in  $N$ -dimensional space with

$$d\mathbf{w} = [dw_1, \dots, dw_N]$$

whose entries are very **tiny**; then, the variation of  $y$  is

$$dy = \dot{f}_1(\mathbf{w}_0) dw_1 + \dots + \dot{f}_N(\mathbf{w}_0) dw_N = \sum_{n=1}^N \dot{f}_n(\mathbf{w}_0) dw_n$$

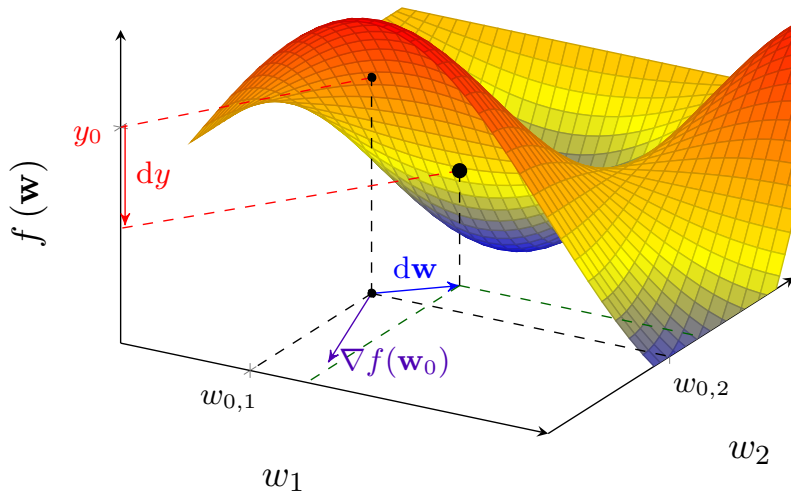
## Review: *Gradient of a Function*

We can use inner-product to represent  $dy$  compactly

$$\begin{aligned} dy &= \sum_{n=1}^N \dot{f}_n(\mathbf{w}_0) dw_n = \underbrace{\begin{bmatrix} \dot{f}_1(\mathbf{w}_0) & \dots & \dot{f}_N(\mathbf{w}_0) \end{bmatrix}}_{\nabla f(\mathbf{w}_0)^T} \begin{bmatrix} dw_1 \\ \vdots \\ dw_N \end{bmatrix} \\ &= \nabla f(\mathbf{w}_0)^T d\mathbf{w} \end{aligned}$$

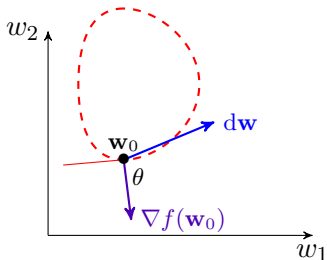
We call  $\nabla f(\mathbf{w}_0)$  *the gradient of  $f(\cdot)$  at  $\mathbf{w} = \mathbf{w}_0$*

## Review: *Gradient of a Function*



## Review: Gradient of a Function

Let's get to the  $\mathbf{w}$ -plane: the **gradient** is perpendicular to the **contour level**<sup>4</sup>



The variation of  $y$  is the inner product of these two vectors

$$dy = \nabla f(\mathbf{w}_0)^T d\mathbf{w} = \|\nabla f(\mathbf{w}_0)\| \|d\mathbf{w}\| \cos(\theta)$$

where  $\|\cdot\|$  is the Euclidean norm, i.e.,  $\|\mathbf{w}\| = \sqrt{w_1^2 + w_2^2}$

<sup>4</sup>Counter level of  $f$  is the set of  $\mathbf{w}$ 's, such that  $f(\mathbf{w}) = \text{constant}$

## Review: *Gradient of a Function*

*Say we move with a tiny step of fixed size: so we have*

$$\|\mathbf{dw}\| = \varepsilon$$

*for some small  $\varepsilon$*

- + *How can we move, such that  $y$  maximally increases?*
- Well, we need  $\theta = 0$  meaning that  
*we should move in the direction of gradient*

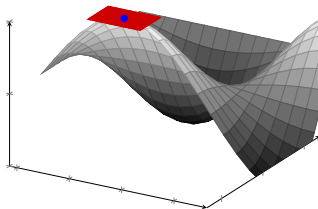
## Moral of Story

The function decreases maximally if  $\theta = \pi$  or

*we move in the direction of negative gradient*

## Review: *Gradient of a Function*

*When do we have zero gradient?* Either when we are at a **maximum**

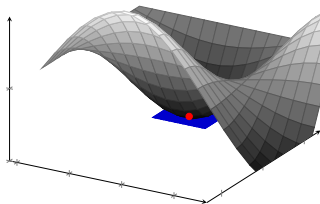


We can again relate it to the *second order derivatives* of the function

at **maximum** *Hessian matrix* is **negative** definite

## Review: *Gradient of a Function*

When do we have zero gradient? or when we are at a **minimum**

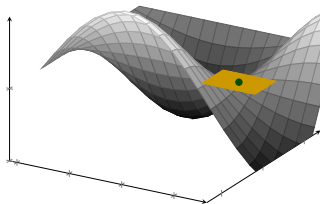


We can again relate it to the *second order derivatives* of the function

at **minimum** *Hessian matrix* is **positive** definite

## Review: *Gradient of a Function*

*When do we have zero gradient? or when we are at a saddle point*



We can again relate it to the *second order derivatives* of the function

at saddle point *Hessian matrix* is neither *negative* nor *positive* definite

## Reminder: *Hessian*

Just as a reminder: **Hessian** is the matrix of all *second order derivatives*

$$\nabla^2 f(\mathbf{w}_0) = \begin{bmatrix} \frac{\partial^2}{\partial w_1^2} f(\mathbf{w}_0) & \frac{\partial^2}{\partial w_1 \partial w_2} f(\mathbf{w}_0) & \dots & \frac{\partial^2}{\partial w_1 \partial w_N} f(\mathbf{w}_0) \\ \vdots & & & \vdots \\ \frac{\partial^2}{\partial w_N \partial w_1} f(\mathbf{w}_0) & \frac{\partial^2}{\partial w_N \partial w_2} f(\mathbf{w}_0) & \dots & \frac{\partial^2}{\partial w_N^2} f(\mathbf{w}_0) \end{bmatrix}$$

We will see that it's pretty expensive to compute, and hence

we **never** use it in practice

## Moral of Story: *Gradient Decent*

- + *What is the whole motive of this discussions?*
- *At any point  $\mathbf{w}$ , if we want to move in a direction that the function reduces we should move with negative of gradient at  $\mathbf{w}$*

So, we can complete our optimization algorithm as follows:

```
1: Initiate at some  $\mathbf{w}^{(0)} \in \mathbb{R}^D$  and deviation  $\Delta = +\infty$ 
2: Choose some small  $\epsilon$  and  $\eta$ , and set  $t = 1$ 
3: while  $\Delta > \epsilon$  do
4:   Update weights as  $\mathbf{w}^{(t)} \leftarrow \mathbf{w}^{(t-1)} - \eta \nabla \hat{R}(\mathbf{w}^{(t-1)})$ 
5:   Update the deviation  $\Delta = |\hat{R}(\mathbf{w}^{(t)}) - \hat{R}(\mathbf{w}^{(t-1)})|$ 
6: end while
```

The scalar  $\eta$  is the step-size we take in each iteration:

*we usually call it learning rate*

## Behavior of Gradient Decent

- + Can we always use gradient descent?
- Pretty much Yes! The problem starts only when *the empirical loss is not differentiable*

*How to handle this problem?*

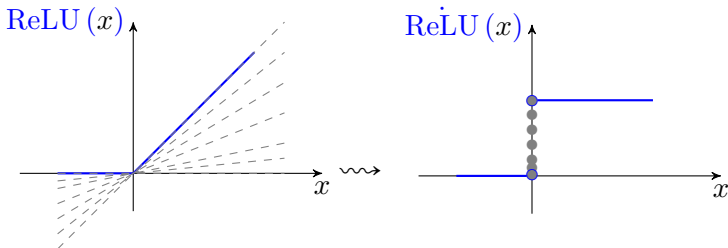
There are two sources for *being non-differentiable*

1. a function that is **continuous** but **not differentiable**
2. a **discontinuous** function

*Let's look at each case separately*

## Behavior of Gradient Decent: *Non-differentiable Elements*

An example of a **non-differentiable** **continuous** function is ReLU

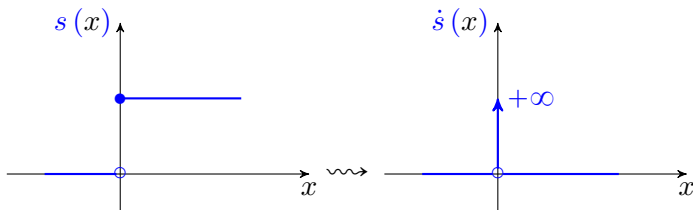


In this case, we define a *sub-gradient* and use it instead of gradient

take the **slope** of a line that lies below the curve at **the point**

## Behavior of Gradient Decent: *Discontinuous Elements*

An example of a **discontinuous** function is the step function



The gradient is somehow **infinite**! *We can only rely on the sign of variation*

*we always **avoid such elements** in our model and loss*

**Bingo!** You may recall that we discouraged the choice of **activation** and **loss function** in the **perceptron** machine

## Behavior of Gradient Decent: *At Stationary Points*

- + Now, let's assume that *we've handled differentiability*. Does gradient decent *always* end up at the minimum point?
- This brings up the concept of *convergence*

*Let's look at the algorithm again*

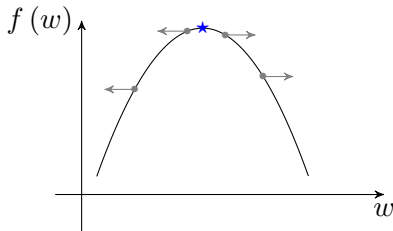
```
1: Initiate at some  $\mathbf{w}^{(0)} \in \mathbb{R}^D$  and deviation  $\Delta = +\infty$   
2: Choose some small  $\epsilon$  and  $\eta$ , and set  $t = 1$   
3: while  $\Delta > \epsilon$  do  
4:   Update weights as  $\mathbf{w}^{(t)} \leftarrow \mathbf{w}^{(t-1)} - \eta \nabla \hat{R}(\mathbf{w}^{(t-1)})$   
5:   Update the deviation  $\Delta = |\hat{R}(\mathbf{w}^{(t)}) - \hat{R}(\mathbf{w}^{(t-1)})|$   
6: end while
```

Intuitively, if we set  $\epsilon$  *very small*: the algorithm stops when the *gradient is close to zero*, i.e., when we are at a *maximum*, *minimum* or an *inflection/saddle-point*

*Let's see how the algorithm behaves when we get close to such point*

## Behavior of Gradient Decent: *At Stationary Points*

When we are around a *maximum*



If we are *exactly* at a *maximum*; then, the algorithm *stops*. But, in reality

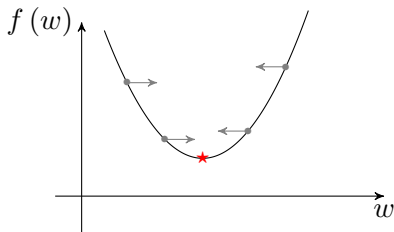
*we land somewhere around it*

at such points, *the algorithm always pushes us outwards*

*Gradient descent practically does not get into a maximum*

## Behavior of Gradient Decent: *At Stationary Points*

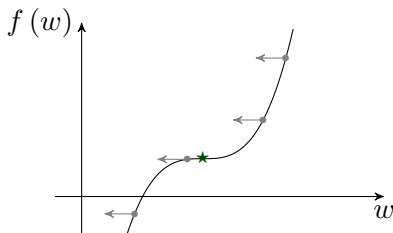
When we are around a *minimum*



Around *minima*, the algorithm always pushes us towards the *minimum*

## Behavior of Gradient Decent: *At Stationary Points*

When we are around an *inflection point*



If we are **exactly** at an *inflection*; then, the algorithm **stops**. But, in reality

*we land somewhere around it*

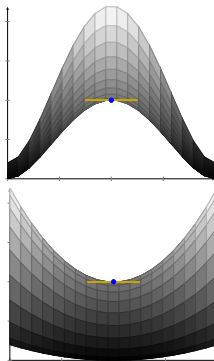
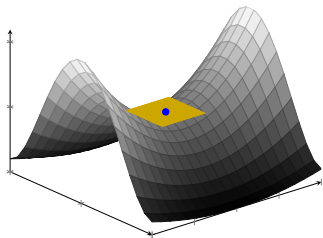
at such points, *the algorithm always pushes us somewhere else*

**Gradient descent** *practically* does not get into an *inflection*

## Behavior of Gradient Decent: *At Stationary Points*

- + Can we extend this conclusion to *saddle-points*?
- Yes, but with a bit of caution!

*At saddle points, function is maximized in a direction and minimized in another*



## Behavior of Gradient Decent: *At Stationary Points*

So, for a *saddle-point* we can conclude: if we are *exactly* at a *saddle-point*; then, the algorithm *stops*. But, in reality

*we land somewhere around it*

If at *that point*, the gradient has a component in the direction that the function is maximized; then, *the algorithm pushes us outwards*.

- + *Can it happen that we do not land at such point?*
- Thinking with an engineering mindset: *Not really!*

So we could say

*Gradient descent almost never gets trapped at a saddle-point*

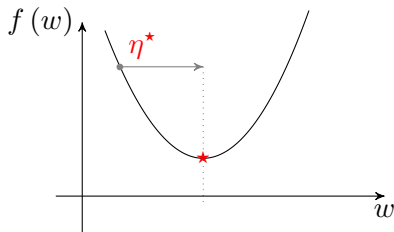
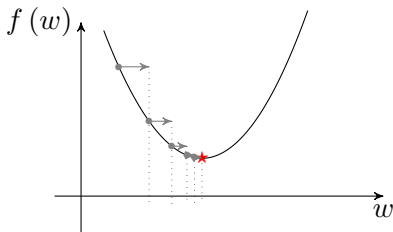
# Behavior of Gradient Decent: *Convergence*

## Moral of Story

*Gradient descent almost never gets trapped at a point that is not minimum*

+ Nice! But, does it always *converge*?

With small *learning rates*, the algorithm *converges*; how small?  $\eta < \eta^*$



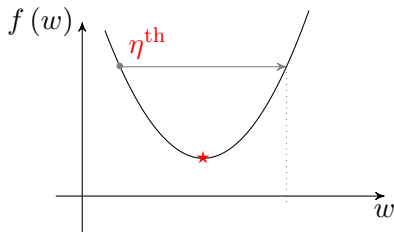
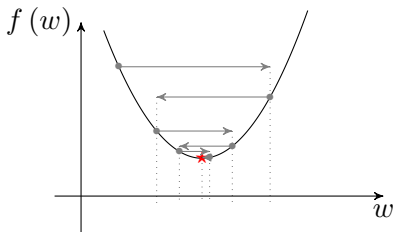
# Behavior of Gradient Decent: *Convergence*

## Moral of Story

*Gradient descent almost never gets trapped at a point that is not minimum*

+ Nice! But, does it always *converge*?

With larger *learning rates*, the algorithm starts *oscillating*:  $\eta^* < \eta < \eta^{\text{th}}$



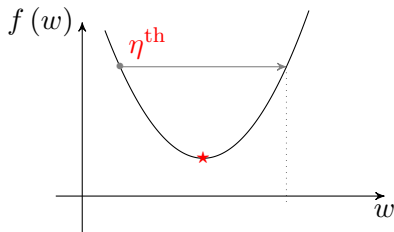
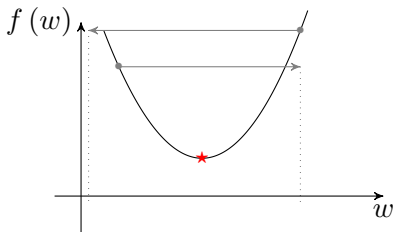
# Behavior of Gradient Decent: *Convergence*

## Moral of Story

*Gradient descent almost never gets trapped at a point that is not minimum*

+ Nice! But, does it always *converge*?

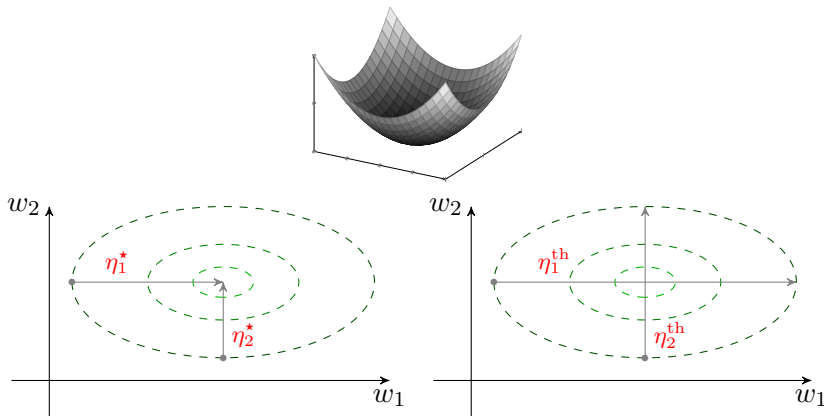
With extremely large *learning rates*, the algorithm *diverges*:  $\eta > \eta^{\text{th}}$



## Behavior of Gradient Decent: *Convergence*

We can extend this idea to the multi-dimensional functions; however,

$\eta^*$  and  $\eta^{\text{th}}$  are *different* on each axis



## Behavior of Gradient Decent: *Convergence*

One may suggest that we use a *vector* of *learning rates*, i.e.,

$$w_n^{(t)} \leftarrow w_n^{(t-1)} - \eta_n \frac{\partial}{\partial w_n} \hat{R}(\mathbf{w}^{(t-1)})$$

for each  $n = 1, \dots, N$ . *This is however not easy; the easier way is to focus on*

$$\eta^\star = \min_n \eta_n^\star \quad \text{and} \quad \eta^{\text{th}} = \min_n \eta_n^{\text{th}}$$

## Behavior of Gradient Decent: *Convergence*

Clearly, there is always a *trade-off*

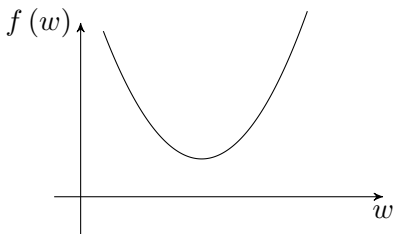
- We can choose a **large learning rate**
    - + *Gradient descent* converges **faster**: *high convergence speed*
    - The chance of *divergence* however **increases**: *high divergence rate*
  - We can choose a **small learning rate**
    - *Gradient descent* converges **slowly**: *low convergence speed*
    - + The chance of *divergence* is now **very low**: *low divergence rate*
- + Well, say we are patient! Then, choosing a small *learning rate* is safe! Right?!
- If empirical risk is **convex**, yes! But, with non-convex risks *not always!*

## Reminder: Convex Function

$f(\cdot) : \mathbb{R}^N \mapsto \mathbb{R}$  is convex, if for any two points  $\mathbf{x}_1, \mathbf{x}_2 \in \mathbb{R}^N$ , we have

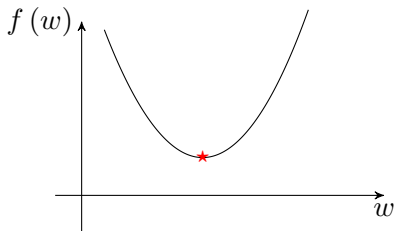
$$f(\lambda \mathbf{x}_1 + (1 - \lambda) \mathbf{x}_2) \leq \lambda f(\mathbf{x}_1) + (1 - \lambda) f(\mathbf{x}_2)$$

for all  $0 \leq \lambda \leq 1$



## Behavior of Gradient Decent: *Convergence*

In convex functions, we *don't have disjoint local minima*



So, if we *choose a small learning rate*

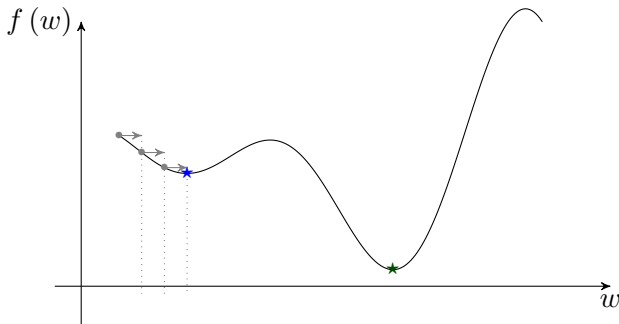
*we surely converge to the global minimum*

## Behavior of Gradient Decent: *Convergence*

*But, almost all empirical risks in deep learning are non-convex, so*

*we have multiple disjoint local minima*

*Gradient descent converges to one of them, but not necessarily the global one*



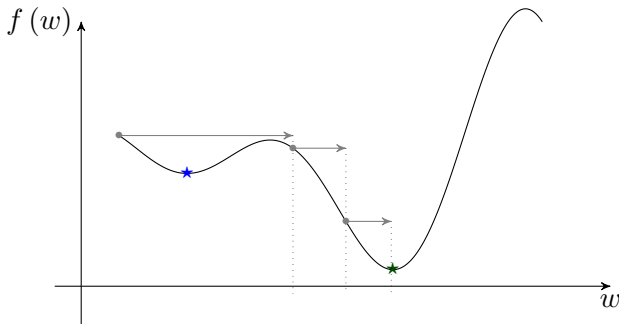
*Too small learning rate can leave us in a bad local minimum!*

## Behavior of Gradient Decent: *Convergence*

*But, most empirical losses in deep learning are non-convex:*

we *have multiple disjoint local minima*

*Gradient descent converges to one of them, but not necessarily the global one*



*Initial larger learning rates might take us out of bad local minima!*

## Behavior of Gradient Decent: *Convergence*

*We can hence conclude a more general trade-off*

- We can choose a **large learning rate**
    - + *Gradient descent* converges **faster**: *high convergence speed*
    - + *Gradient descent* may fall out of a **local minimum**: *lower risk*
    - The chance of **divergence** however **increases**: *high divergence rate*
  - We can choose a **small learning rate**
    - *Gradient descent* converges **slowly**: *low convergence speed*
    - *Gradient descent* traps in a **local minimum**: *high risk*
    - + The chance of **divergence** is now **very low**: *low divergence rate*
- + *How do we do it then?*
- In practice, we need to **schedule** it

*We will talk about this more!*

## Gradient Decent: *Summary*

- 1: Initiate at some  $\mathbf{w}^{(0)} \in \mathbb{R}^D$  and deviation  $\Delta = +\infty$
- 2: Choose some small  $\epsilon$  and  $\eta$ , and set  $t = 1$
- 3: **while**  $\Delta > \epsilon$  **do**
- 4:   Update weights as  $\mathbf{w}^{(t)} \leftarrow \mathbf{w}^{(t-1)} - \eta \nabla \hat{R}(\mathbf{w}^{(t-1)})$
- 5:   Update the deviation  $\Delta = |\hat{R}(\mathbf{w}^{(t)}) - \hat{R}(\mathbf{w}^{(t-1)})|$
- 6: **end while**

*Gradient descent converges almost always to a local minimum*

- With convex empirical risks, this is *global minimum*
- With non-convex empirical risks, this is not necessarily *global minimum*
- *Learning rate* is a crucial parameter that tunes the *convergence*

*There are other optimization algorithms that work based on gradient: we will talk about them later!*



UNIVERSITY OF  
TORONTO

## Further Read

## Further Read

### Read More on These Topics



Goodfellow, et al., *Deep Learning*

↳ [Chapter 4](#)



Bishop & Bishop, *Deep Learning: Foundations and Concepts*

↳ [Chapters 6 and 7](#)

### Nice to Check

1. *Perceptron Simulation Experiments*

↳ F. Rosenblatt (1960)

2. *Approximation by superpositions of a sigmoidal function*

↳ G. V. Cybenko (1989)